v.0.0.1

최병권

NUTRI CHECK MANAGER

Contents

1. Introduce to Program

1-1. 기획의도

1-2. 소프트웨어 구성

2. Details

2-1. 메인 화면

2-1-1. 오늘의 복용 현황

2-1-2. 재구매 임박 목록

2-1-3. 구현

2-2. 영양제 관리

2-2-1. 영양제 등록

2-2-2. 영양제 목록

2-2-3. 영양제 상세 조회

2-2-4. 영양제 정보 수정

2-2-5. 영양제 검색

2-3. 복용 내역 관리

2-3-1. 복용 내역 목록

3. Function

Introduce to Program

"

오늘 영양제를 먹었었나?

건강관리 필수 아이템인 '영양제'
바쁜 현대인들에겐 영양제를 챙겨 먹는 것 조차 일이 되곤 한다.
까먹고, 더 먹고, 넘기고..... 이런 불상사를 막고자 만든 프로그램
「 NUTRI CHECK MANAGER 」와 함께 바쁜 일상 속
잊지 말고 건강 챙기기! 도전해보자!

1-2. 소프트웨어 구성

- JAVA
- Spring Boot
- JPA
- Postgres

- Vue.js
- Javascript
- Nuxt

Details

2-1-1. 오늘의 복용 현황_Java_Controller(pillController)

```java
@Api(tags = "영양제 세부 관리")
@RestController
@RequiredArgsConstructor
@RequestMapping("/v1/pill")
public class PillController {
    6 usages
    private final PillService pillService;

@ApiOperation(value = "영양제 재구매 임박 리스트")
@GetMapping("/all/repurchase")
public ListResult<PillRepurchaseItem> getRepurchase() {
    return ResponseService.getListResult(pillService.getRepurchases(), isSuccess: true);
}
```

- Final을 사용하여 필수적으로 pillService와 같이 연동해야함을 알려줍니다.
- ApiOperation을 사용하여 Swagger에 표시될 내용을 입력해줍니다.

- PillRepurchaseItem을 List에 담아옵니다
- ResponseService(응답서비스)를 통해 Repurchases() 데이터들을 가져오면 성공하였습니다! 라고 알려줍니다.

2-1-2. 재구매 임박 목록(1)_Java_Model(pillRepurchaseItem)

```java
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class PillRepurchaseItem {
    1 usage
    @ApiModelProperty(notes = "영양제 순번")
    private Long id;

    1 usage
    @ApiModelProperty(notes = "영양제 영양 정보")
    private String pillInfoName;

    1 usage
    @ApiModelProperty(notes = "영양제 정보")
    private String pillAllNames;

    1 usage
    @ApiModelProperty(notes = "영양제 가격")
    private Integer pillPrice;

    1 usage
    @ApiModelProperty(notes = "남은 영양제 개수")
    private Integer remainingQuantity;

    1 usage
    private PillRepurchaseItem(pillRepurchaseItemBuilder builder) {
        this.id = builder.id;
        this.pillInfoName = builder.pillInfoName;
        this.pillAllNames = builder.pillAllNames;
        this.pillPrice = builder.pillPrice;
        this.remainingQuantity = builder.remainingQuantity;
    }
```

- NoArgsConstructor를 사용해서 내부 생성자만 사용할 수 있도록 합니다.
- ApiModelProperty를 사용하여 swagger에 표시될 정보들을 기입합니다.

- PillRepurchaseItem을 private를 사용하여 Service와 Controller 사이에서만 사용할 수 있게 합니다.
- Builder를 생성하여 Pill정보들을 받아옵니다.

2-1-2. 재구매 임박 목록(2)_Java_Model(pillRepurchaseItem)

```java
public static class pillRepurchaseItemBuilder implements CommonModelBuilder<PillRepurchaseItem> {
    2 usages
    private final Long id;
    2 usages
    private final String pillInfoName;
    2 usages
    private final String pillAllNames;
    2 usages
    private final Integer pillPrice;
    2 usages
    private final Integer remainingQuantity;

    1 usage
    public pillRepurchaseItemBuilder(Pill pill) {
        this.id = pill.getId();
        this.pillInfoName = pill.getPillInfo().getName();
        this.pillAllNames = pill.getPillCompany() + " / " + pill.getPillName();
        this.pillPrice = pill.getPillPrice();
        this.remainingQuantity = pill.getRemainingQuantity();
    }

    @Override
    public PillRepurchaseItem build() { return new PillRepurchaseItem( builder: this); }
}
```

- PillRepurchaseBuilder를 통해 id, pillInfoName, pillAllNames, pillPrice, remainingQuantity 를 받아옵니다.
- this를 사용해 pill(영양제)정보를 받아옵니다.

- Pill정보 : 제조사, 이름, 가격, 재고수량
- Build에 필요한 Override를 해줍니다.

2-1-3. 오늘의 복용 현황, 재구매 임박 목록_Vue

< Template

```
<template>
    <div>
        <h1 class="custom-title-h1"> ▶ 오늘의 복용 현황</h1>
        <br />
        <div>
            <el-table
                :data="list"
                stripe
                style="width: 100%">
                <el-table-column
                    prop="pillFullName"
                    label="영양제 이름 및 제조사">
                </el-table-column>
                <el-table-column
                    prop="dosesCount"
                    label="권장 복용량">
                </el-table-column>
                <el-table-column
                    prop="realDosesCount"
                    label="실제 복용량">
                </el-table-column>
            </el-table>
        </div>
        <br />
        <br />
        <h1 class="custom-title-h1"> ▶ 재구매 임박 영양제</h1>
        <br />
        <div>
            <el-table
                :data="listRepurchase"
                stripe
                style="width: 100%">
                <el-table-column
                    prop="nutriInfoName"
                    label="영양 정보 이름">
                </el-table-column>
                <el-table-column
                    prop="pillFullName"
                    label="영양제 이름 및 제조사">
                </el-table-column>
                <el-table-column
                    prop="pillPrice"
                    label="영양제 가격">
                </el-table-column>
                <el-table-column
                    prop="remainingQuantity"
                    label="잔여 수량">
                </el-table-column>
            </el-table>
        </div>
    </div>
</template>
```

Script Method >

```
<script>
export default {
    data() {
        return {
            list: [],
            listRepurchase: []
        }
    },
    methods: {
        getTodayEatList() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            this.$store.dispatch(this.$todayPilEatStatusConstants.DO_TODAY_PILL_EAT_STATUS_LIST)
                .then((res) => {
                    this.list = res.data.list
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        },
        getRepurchaseList() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            this.$store.dispatch(this.$pillConstants.DO_REPURCHASE_LIST)
                .then((res) => {
                    this.listRepurchase = res.data.list
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        }
    },
    created() {
        this.getTodayEatList()
        this.getRepurchaseList()
    },
    mounted() {
        this.$store.commit(this.$menuConstants.FETCH_SELECTED_MENU, 'DASH_BOARD')
    },
}
</script>
```

2-1-3. 오늘의 복용 현황, 재구매 임박 영양제_Vue(구현화면)

님

- HOME
- 대시보드
- 영양제 관리
- 영양제 목록
- 복용내역 관리
- 복용내역 목록

HOME > 대시보드

▶ 오늘의 복용 현황

영양제 이름 및 제조사	권장 복용량	실제 복용량
대광약품 관절엔 콘드로이친 1200 900mg	1	0
순수식품 로얄 프로폴리스 플라보노이드 17 900 mg	2	0
락토핏 장튼튼 유산균 100억가지	2	0
뉴트리원 지아잔틴 164 500mg	1	0
고려은단 퓨어 알티지 r200	3	0

▶ 재구매 임박 영양제

영양 정보 이름	영양제 이름 및 제조사	영양제 가격	잔여 수량
철분	관절엔 콘드로이친 1200 900mg 대광약품	28710	5

2-2-1. 영양제 등록_Java_PillRequest(model)

```java
@Getter
@Setter
public class PillRequest {

    no usages
    @ApiModelProperty(notes = "영양제 이름 (1~35자)", required = true)
    @NotNull
    @Length(min = 1, max = 35)
    private  String pillName;

    no usages
    @ApiModelProperty(notes = "영양제 제조사 (1~20 자)", required = true)
    @NotNull
    @Length(min = 1, max = 20) // string은 길이조절을 해야해서 min,max lenght를 이용해서 사용
    private  String pillCompany;

    no usages
    @ApiModelProperty(notes = "영양제 가격 (0 이상)", required = true)
    @NotNull
    @Min(value = 0) // integer은 값 자체의 숫자의 범위를 지정하는거라 바로 min,max씀
    private  Integer price;

    no usages
    @ApiModelProperty(notes = "영양 정보", required = true)
    @NotNull
    @Enumerated(value = EnumType.STRING)
    private  NutriInfo nutriInfo;

    no usages
    @ApiModelProperty(notes = "영양제 복용시작일자 (yyyy-mm-dd)", required = true)
    @NotNull
    private  LocalDate startDate;

    no usages
    @ApiModelProperty(notes = "영양제 유통기한 (yyyy-mm-dd)", required = true)
    @NotNull
    private  LocalDate expiationDate;
```

```java
    @ApiModelProperty(notes = "영양제 총 수량(1회 기준) (1이상)", required = true)
    @NotNull
    @Min(value = 1)
    private Integer totalQuantity;

    no usages
    @ApiModelProperty(notes = "섭취 주기 여부", required = true)
    @NotNull
    private  Boolean isDaily;

    no usages
    @ApiModelProperty(notes = "1회 복용 횟수 (1 이상)", required = true)
    @NotNull
    @Min(value = 1)
    private  Integer oneTimeQuantity;

}
```

- DB에 저장할 목적으로 영양제 정보를 수집하기 위해 PillRequest를 다음과 같이 구성 했습니다.
- Enums를 활용하여 영양 정보 생성하였고 EnumType을 String으로 하여 추후 보완 및 저장에 용이하도록 했습니다.
- 입력 값 오류를 막기 위한 Valid 사용을 위해 NotNull, Length, Min, Max등 기준 값을 지정해주었습니다.
- Swagger 및 코드 내에 매개변수의 이름,필수 값 등을 지정하기 위해 ApiModelProperty를 활용했습니다.

2-2-1. 영양제 등록_Java_PillService+PillController

```java
@Service
@RequiredArgsConstructor
public class PillService {

    7 usages
    private final PillRepository pillRepository;

    SO WOO LEE
    public void setPill(PillRequest request) {
        LocalDate today = LocalDate.now(); // today 변수 만들기

        //오늘이 받을 값보다 나중(after)이면 true
        if(today.isAfter(request.getExpiationDate())) throw new CDataPastException();
        Pill addData = new Pill.PillBuilder(request).build();

        pillRepository.save(addData);
    }
```

- Repository에 JPA를 구현 및 가동하였고 RequiredArgsConstructor 활용하여 함께 쓰이도록 했습니다.
- 영양정보를 가져와 setting해주는 method를 생성하였습니다.
- CDataPastException 생성으로 영양제 유통기한 날짜가 지난 정보 입력 시 아래와 같은 메시지를 띄우게 했습니다.

```java
, DATE_PAST( code: -20000, msg: "유통기한이 지난 제품입니다.")
```

- Pill entity에서 활용한 builde를 통해 번거로운 sette작업을 없애 실수를 줄여 DB저장에 정확도를 높였습니다.

```java
@Api(tags = "영양제 목록")
@RestController
@RequiredArgsConstructor
@RequestMapping("/v1/pill")
public class PillController {

    6 usages
    private final PillService pillService;

    no usages  SO WOO LEE
    @ApiOperation(value = "영양제 등록")
    @PostMapping("/data")
    public CommonResult setPill(@RequestBody @Valid PillRequest request) {
        pillService.setPill(request);

        return ResponseService.getSuccessResult();
    }
```

- Api를 사용해 Swagger의 가독성을 높였습니다.
- Body로 사용자에게 값을 받기 위해 PostMapping을 사용했고 RequestBody의 오류 범실을 줄이기 위해 Valid를 썼습니다.
- ResponseService를 따로 생성하여 결과 값을 아래와 같이 지정해주어 return값의 질을 높였습니다.

```java
SUCCESS( code: 0, msg: "성공하였습니다.")
, FAILED( code: -1, msg: "실패하였습니다.")
```

2-2-1. 영양제 등록(1)_Vue

```
<template>
    <div>
        <div>
            <h1 class="custom-title-h1">▶ 영양제 등록 </h1>
        </div>
        <div style="text-align: center">
            <el-form :model="ruleForm" :rules="rules" ref="ruleForm" label-width="120px" class="demo-
ruleForm">
                <br />
                <br />
                <el-form-item label="영양제 이름" prop="pillName">
                    <el-input v-model="ruleForm.pillName"></el-input>
                </el-form-item>
                <br />
                <br />
                <el-form-item label="영양제 제조사" prop="pillCompany">
                    <el-input v-model="ruleForm.pillCompany"></el-input>
                </el-form-item>
                <br />
                <br />
                <el-form-item label="영양제 가격" prop="price">
                    <el-input v-model="ruleForm.price"></el-input>
                </el-form-item>
                <br />
                <br />
                <el-form-item label="영양 정보" prop="nutriInfo">
                    <el-select v-model="ruleForm.nutriInfo" placeholder="영양정보를 선택해주세요">
                        <el-option label="루테인" value="LUTEIN"></el-option>
                        <el-option label="철분" value="IRON"></el-option>
                        <el-option label="미네랄" value="MINERALS"></el-option>
                        <el-option label="오메가3" value="OMEGA3"></el-option>
                        <el-option label="프로폴리스" value="PROPOLIS"></el-option>
                        <el-option label="프로바이오틱스" value="PROBIOTICS"></el-option>
                        <el-option label="칼슘" value="CALCIUM"></el-option>
                    </el-select>
                </el-form-item>
                <br />
                <br />
                <el-row>
                    <el-col :span="11">
                        <el-form-item label="복용시작일자" required>
                            <el-form-item prop="startDate">
                                <el-date-picker type="date" placeholder="Pick a date" v-
model="ruleForm.startDate"
                                style="width: 100%;"></el-date-picker>
                            </el-form-item>
                        </el-form-item>
                    </el-col>
                    <el-col :span="11">
                        <el-form-item label="유통기한" required>
                            <el-form-item prop="expiationDate">
                                <el-date-picker type="date" placeholder="Pick a date" v-
model="ruleForm.expiationDate" style="width: 100%;"></el-date-picker>
                            </el-form-item>
                        </el-form-item>
                    </el-col>
                </el-row>
                <br />
                <br />
                <el-row>
                    <el-col :span="8">
                        <el-form-item>
                            <el-switch
                                v-model="ruleForm.isDaily"
                                active-text="일간 섭취"
                                inactive-text="주간 섭취">
                            </el-switch>
                        </el-form-item>
                    </el-col>
                    <el-col :span="8">
                        <el-form-item label="영양제 총 수량" prop="totalQuantity">
                            <el-input-number v-model="ruleForm.totalQuantity" :min="1"></el-input-
number>
                        </el-form-item>
                    </el-col>
                    <el-col :span="8">
                        <el-form-item label="1회 복용 횟수" prop="oneTimeQuantity">
                            <el-input-number v-model="ruleForm.oneTimeQuantity" :min="1"></el-input-
number>
                        </el-form-item>
                    </el-col>
                </el-row>
                <br />
                <br />
                <br />
                <br />
                <el-form-item>
                    <el-button type="primary" @click="doSetData">등록</el-button>
                    <el-button>취소</el-button>
                </el-form-item>
            </el-form>

        </div>
    </div>
</template>
```

< Template

2-2-1. 영양제 등록(2)_Vue

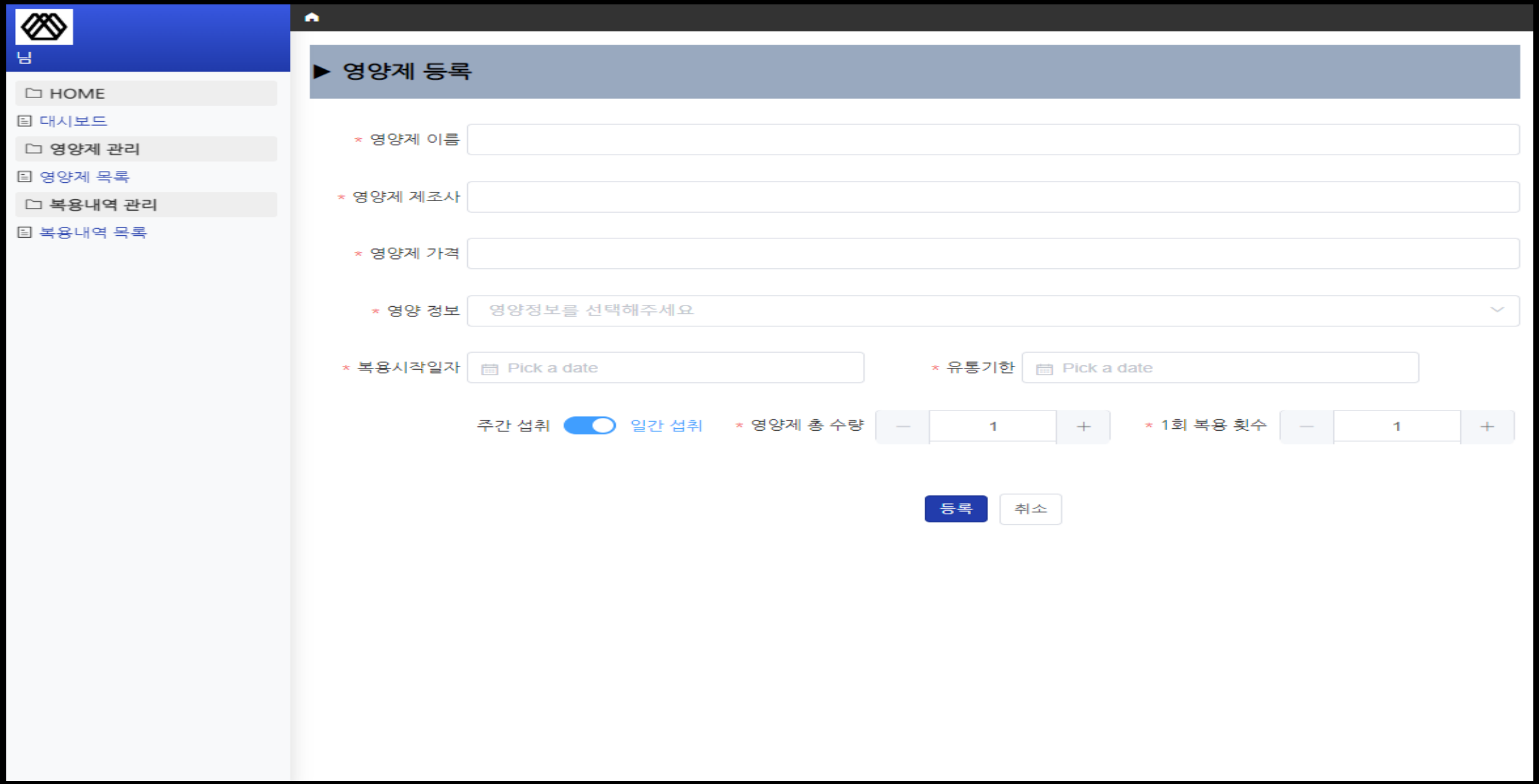
```
<script>
export default {
    data() {
        return {
            ruleForm: {
                pillName: '',
                pillCompany: '',
                price: '',
                nutriInfo: '',
                startDate: '',
                expiationDate: '',
                isDaily: true,
                totalQuantity: '',
                oneTimeQuantity: ''
            },
            rules: {
                pillName: [
                    { required: true, message: '필수값입니다.', trigger: 'blur' },
                    { min: 1, max: 35, message: '1~35자 내로 작성', trigger: 'blur' }
                ],
                pillCompany: [
                    { required: true, message: '필수값입니다', trigger: 'blur' },
                    { min: 1, max: 20, message: '1~20자 내로 작성 ', trigger: 'blur' }
                ],
                price: [
                    { required: true, message: '필수값입니다', trigger: 'blur' },
                    { min: 0, message: '0 이상 작성', trigger: 'blur' }
                ],
                nutriInfo: [
                    { required: true, message: '필수값입니다', trigger: 'change' }
                ],
                startDate: [
                    { type: 'date', required: true, message: '필수값입니다', trigger: 'change' }
                ],
                expiationDate: [
                    { type: 'date', required: true, message: '필수값입니다', trigger: 'change' }
                ],
                totalQuantity: [
                    { required: true, message: '필수값입니다', trigger: 'blur' },
                ],
                oneTimeQuantity: [
                    { required: true, message: '필수값입니다', trigger: 'blur' },
                ]

            }
        };
    },
```

< Script Data

Script Method >

```
    methods: {
        doSetData() {
            let formName = 'ruleForm'
            this.$refs[formName].validate(valid => {
                if (valid) {
                    this.setData()
                } else {
                    return false
                }
            })
        },
        setData() {
            let payload = {
                pillName: this.ruleForm.pillName,
                pillCompany: this.ruleForm.pillCompany,
                price: Number(this.ruleForm.price),
                nutriInfo: this.ruleForm.nutriInfo,
                startDate: this.ruleForm.startDate,
                expiationDate: this.ruleForm.expiationDate,
                isDaily: this.ruleForm.isDaily,
                totalQuantity: this.ruleForm.totalQuantity,
                oneTimeQuantity: this.ruleForm.oneTimeQuantity
            }
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            this.$store.dispatch(this.$pillConstants.DO_CREATE, payload)
                .then(res => {
                    this.$toast.success(res.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                    this.$router.replace(`/pill/all-list`)
                })
                .catch(err => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        }
    }
}
</script>

<style scoped>

</style>
```

2-2-1. 영양제 등록_ Vue(구현화면)



2-2-2. 영양제 목록_Java_PillItem(model)

```java
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class PillItem {

    1 usage
    @ApiModelProperty(notes = "시퀀스")
    private Long id;
    1 usage
    @ApiModelProperty(notes = "영양제 이름")
    private String pillName;
    1 usage
    @ApiModelProperty(notes = "영양제 제조사")
    private String pillCompany;
    1 usage
    @ApiModelProperty(notes = "영양제 가격")
    private Integer pillPrice;
    1 usage
    @ApiModelProperty(notes = "현재 복용 여부")
    private String isEat;

    1 usage   SO WOO LEE
    private PillItem(PillItemBuilder builder){
        this.id = builder.id;
        this.pillName = builder.pillName;
        this.pillCompany = builder.pillCompany;
        this.pillPrice = builder.pillPrice;
        this.isEat = builder.isEat;
    }

    3 usages   SO WOO LEE
    public static class PillItemBuilder implements CommonModelBuilder<PillItem> {

        2 usages
        private final Long id;
        2 usages
        private final String pillName;
        2 usages
        private final String pillCompany;
        2 usages
        private final Integer pillPrice;
        2 usages
        private final String isEat;
```

```java
        public PillItemBuilder(Pill pill) {
            this.id = pill.getId();
            this.pillName = pill.getPillName();
            this.pillCompany = pill.getPillCompany();
            this.pillPrice = pill.getPrice();
            this.isEat = pill.getIsEat()? "O" : "X";
        }

        SO WOO LEE
        @Override
        public PillItem build() { return new PillItem( builder: this); }
    }
```

- 영양제 목록에서 보여 줄 항목들을 지정하기 위해 PillItem이라는 이름으로 model을 생성했습니다.
  - 현재 복용 중인 영양제를 기준으로 영양제 이름, 제조사, 가격을 보여줄 예정
- NoArgsConstructor을 사용하여 외부에서 생성자를 만들 수 없도록 AccessLevel.Protected를 지정해줍니다.
- Builder패턴을 활용하여 오류 최소화 및 프로그램 실행 속도를 증가시켜줍니다.

2-2-2. 영양제 목록_Java_PillService+PillController

```java
@Service
@RequiredArgsConstructor
public class PillService {

    7 usages
    private final PillRepository pillRepository;

    SO WOO LEE
    public void setPill(PillRequest request) {...}

    1 usage   SO WOO LEE
    public PillResponse getPill(long id) {...}
    1 usage   SO WOO LEE
    public ListResult<PillItem> getPills() {
        List<Pill> originList = pillRepository.findAll();

        List<PillItem> result = new LinkedList<>();

        originList.forEach(item -> result.add(new PillItem.PillItemBuilder(item).build()));

        return ListConvertService.settingResult(result);
    }
```

- 항목이 담긴 PillItem을 List<>를 사용하여 리스트화 하기 위해 getPills라는 이름의 method를 생성해주었습니다.
- 재가공하기 전, 전체 리스트를 먼저 생성시켜야 하므로 LinkedList를 활용하여 리스트를 담을 공간인 result를 지정해주었습니다.
- forEach문을 사용하여 원본데이터를 하나씩 builder에게 재가공한 뒤 만들어둔 result안에 저장되도록 하였습니다.

```java
@Api(tags = "영양제 목록")
@RestController
@RequiredArgsConstructor
@RequestMapping("/v1/pill")
public class PillController {

    6 usages
    private final PillService pillService;

    no usages   new *
    @ApiOperation(value = "영양제 내역 목록")
    @GetMapping("/all")
    public ListResult<PillItem> getPills() {

        return ResponseService.getListResult(pillService.getPills(), isSuccess: true);
    }
```

- Api를 통한 이름 및 값을 명시하여 Swagger 및 코드 작성에 가독성을 높였습니다.
- PillService에서 값을 받아와야 작동할 수 있도록 해주었습니다.
- ListResult를 활용하여 결과 값이 오른쪽과 같이 도출될 수 있게 지정해주었습니다.

```java
private List<T> list;
no usages
private Long totalItemCount;
no usages
private Integer totalPage;
no usages
private Integer currentPage;
```

2-2-2. 영양제 목록_Vue

```vue
<template>
    <div>
        <div>
            <h1 class="custom-title-h1">▶ 영양제 목록 </h1>
        </div>
        <br />
        <div style="text-align: right">
            <el-button type="success" circle @click.native="moveCreate()">등록</el-button>
        </div>
        <br />
        <el-row>
            <el-col :span="12" v-for="(item, index) in list" v-bind:key="index">
                <el-card class="box-card">
                    <div slot="header" class="clearfix">
                        <span>{{ item.pillName }}</span>
                        <el-button style="float: right; padding: 3px 0" type="text"
@click.native="moveDetail(item.id)">상세보기</el-button>
                    </div>
                    <div class="text item">
                        제조사 : {{ item.pillCompany }}
                    </div>
                    <div class="text item">
                        가격 : {{ item.pillPrice }} 원
                    </div>
                    <div class="text item">
                        현재 복용상태 : {{ item.isEat }}
                    </div>
                </el-card>
            </el-col>
        </el-row>
    </div>
</template>
```

> Template

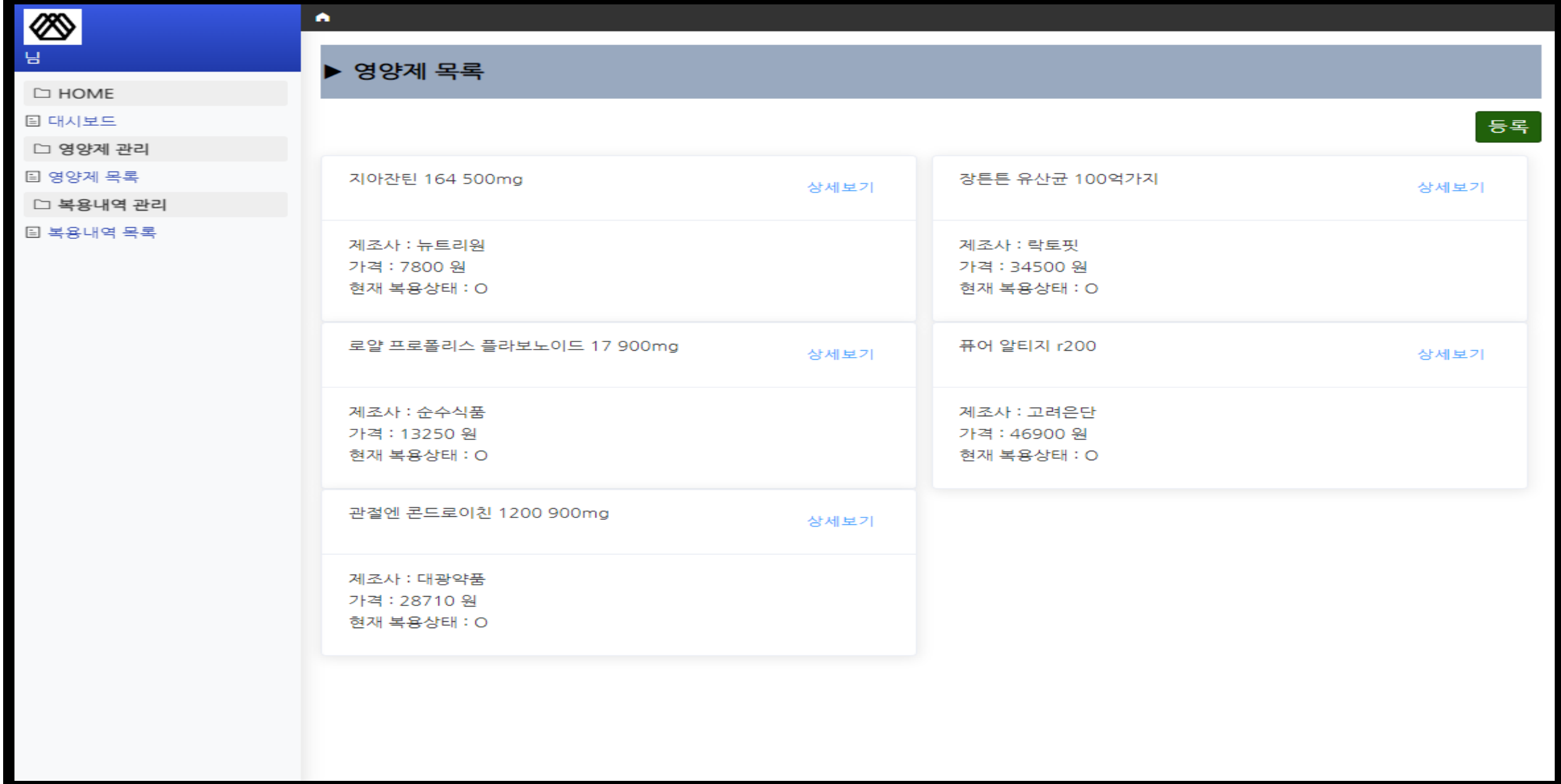
```vue
<script>
export default {
    data() {
        return {
            list: []
        }
    },
    methods: {
        getList() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            this.$store.dispatch(this.$pillConstants.DO_ALL_LIST)
                .then((res) => {
                    this.list = res.data.list
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        },
        delData(id) {
            if (confirm('정말 삭제하시겠습니까?')) { // javascript confirm 검색하기
                this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
                this.$store.dispatch(this.$testDataConstants.DO_DELETE, {id: id})
                    .then((res) => {
                        this.$toast.success(res.data.msg)
                        this.getList()
                        this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                    })
                    .catch((err) => {
                        this.$toast.error(err.response.data.msg)
                        this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                    })
            }
        },
        moveCreate() {
            this.$router.push(`/pill/form`)
        },
        moveDetail(id) {
            let linkBaseUrl = '/pill/detail/'
            this.$router.push(`${linkBaseUrl}${id}`)
        },
        moveEdit(id) {
            let linkBaseUrl = '/customer/edit/'
            this.$router.push(`${linkBaseUrl}${id}`)
        }
    },
    created() {
        this.getList()
    },
    mounted() {
        this.$store.commit(this.$menuConstants.FETCH_SELECTED_MENU, 'MEMBER_LIST')
    },

}
```

> Script_method

```vue
</script>

<style>
.text {
    font-size: 14px;
}

.item {
    margin-bottom: 10px;
}

.clearfix:before,
.clearfix:after {
    display: table;
    content: "";
}
.clearfix:after {
    clear: both
}

.box-card {
    width: 450px;
}
</style>
```

> style

2-2-2. 영양제 목록_Vue(구현화면)



2-2-3. 영양제 정보 상세 조회_Java_PillResponse

```java
@ApiModelProperty(notes = "영양제 시퀀스")
private Long id;

1 usage
@ApiModelProperty(notes = "제조사")
private String pillCompany;

1 usage
@ApiModelProperty(notes = "영양제")
private String pillName;

1 usage
@ApiModelProperty(notes = "가격")
private String pillPriceName;

1 usage
@ApiModelProperty(notes = "영양정보")
private String nutriInfoName;

1 usage
@ApiModelProperty(notes = "복용시작일자 (yyyy-mm-dd)")
private LocalDate startDate;

1 usage
@ApiModelProperty(notes = "유통기한 (yyyy-mm-dd)")
private LocalDate expirationDate;

1 usage
@ApiModelProperty(notes = "총 수량")
private Integer totalQuantity;

1 usage
@ApiModelProperty(notes = "섭취주기")
private Boolean isDaily; // 수정창으로 보낸다

1 usage
@ApiModelProperty(notes = "섭취주기 한글명")
private String isDailyName; // 상세페이지로 보낸다
```

```java
public PillResponseBuilder(Pill pill) {
    this.id = pill.getId();
    this.pillCompany = pill.getPillCompany();
    this.pillName = pill.getPillName();
    this.pillPriceName = pill.getPillPrice() + "원";
    this.nutriInfoName = pill.getNutriInfo().getName();
    this.startDate = pill.getStartDate();
    this.expirationDate = pill.getExpirationDate();
    this.totalQuantity = pill.getTotalQuantity();
    this.isDaily = pill.getIsDaily();
    this.isDailyName = pill.getIsDaily()? "일간" : "주간";
    this.oneTimeQuantity = pill.getOneTimeQuantity();
    this.remainingQuantity = pill.getRemainingQuantity();
    this.isEat = pill.getIsEat();
    this.isEatName = pill.getIsEat()? "복용중" : "복용하지 않음";
}
```

- 영양제 정보를 상세 조회할 수 있는 페이지입니다.
- Boolean 타입은 Boolean과 String 두가지 타입으로 나눴습니다.
- Boolean은 정보 수정 시, switch에 기존에 등록되어 있던 값을 전달할 때 사용하며
- String은 사용자가 해당 Boolean의 값이 어떤 뜻인지 알아보기 쉽도록 추가했습니다.

2-2-3. 영양제 정보 상세 조회(1)_Vue

```
<template>
    <div>
        <div v-if="detailInfo != null">
            <el-descriptions class="margin-top" title="상세내역" :column="3" :size="size" border>
                <template slot="extra">
                    <el-button type="primary" size="small" @click.native="moveEdit(id)">수정</el-button>
                    <el-button size="small" @click.native="moveList()">뒤로</el-button>
                </template>
                <el-descriptions-item>
                    <template slot="label">
                        <i class="el-icon-orange"></i>
                        영양제
                    </template>
                    {{ detailInfo.pillName }}
                </el-descriptions-item>
                <el-descriptions-item>
                    <template slot="label">
                        <i class="el-icon-office-building"></i>
                        제조사
                    </template>
                    {{ detailInfo.pillCompany }}
                </el-descriptions-item>
                <el-descriptions-item>
                    <template slot="label">
                        <i class="el-icon-coin"></i>
                        가격
                    </template>
                    {{ detailInfo.pillPriceName }}
                </el-descriptions-item>
```

< Template

2-2-3. 영양제 정보 상세 조회(2)_Vue

```
<script>
export default {
    asyncData({params}) {
        return {
            id: params.id
        }
    },
    validate({params}) {
        return /^\d+$/.test(params.id)
    },
    data() {
        return {
            id: null,
            detailInfo: null,
            size: ''
        }
    },
    methods: {
        async getDetail() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            await this.$store.dispatch(this.$pillConstants.DO_DETAIL, {id: this.id})
                .then((res) => {
                    this.detailInfo = res.data.data
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        },
        moveEdit(id) {
            let linkBaseUrl = '/pill/edit/'
            this.$router.push(`${linkBaseUrl}${id}`)
        },
        moveList() {
            this.$router.push(`/pill/list`)
        }
    },
    created() {
        this.getDetail()
    },
    mounted() {
        this.$store.commit(this.$menuConstants.FETCH_SELECTED_MENU, 'MEMBER_DETAIL')
    },
}
</script>
```

```
<div v-if="detailInfo != null">
    <el-descriptions class="margin-top" title="상세내역" :column="3" :size="size" border>
        <template slot="extra">
            <el-button type="primary" size="small" @click.native="moveEdit(id)">수정</el-button>
            <el-button size="small" @click.native="moveList()">뒤로</el-button>
        </template>
        <el-descriptions-item>
```

- 영양제 정보를 상세 조회할 수 있는 페이지입니다.
- 가독성을 위해 el-descriptions를 사용해 표로 만들었습니다.

2-2-3. 영양제 정보 상세 조회 _ Vue(구현화면)

상세내역

수정 뒤로

영양제	루테인 1000	제조사	가상제약	가격	10000원
영양정보	루테인	복용시작일자	2023-02-10	유통기한	2023-12-31
총 수량	50	섭취주기	일간	1회 복용량	2
잔여량	10	복용여부	복용중		

2-2-4. 영양제 정보 수정_Java_PillInfoUpdateRequest

```java
public class PillInfoUpdateRequest {
    no usages
    @ApiModelProperty(notes = "섭취주기", required = true)
    @NotNull
    private Boolean isDaily;

    no usages
    @ApiModelProperty(notes = "1회 복용량 (1 이상)", required = true)
    @NotNull
    @Min(value = 1)
    private Integer oneTimeQuantity;

    no usages
    @ApiModelProperty(notes = "잔여량", required = true)
    @NotNull
    @Min(value = 0)
    private Integer remainingQuantity;

    no usages
    @ApiModelProperty(notes = "복용여부", required = true)
    @NotNull
    private Boolean isEat;
}
```

- 등록되어 있는 영양제 정보를 수정하는 기능입니다.
- Vaild를 사용해 필수로 입력받아야 하는 값에 @Notnull을 달아두었습니다.

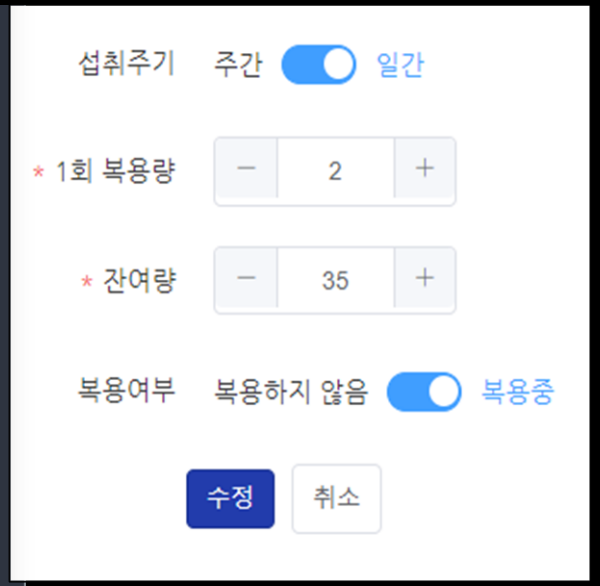
2-2-4. 영양제 정보 수정_Vue

```
<template>
    <div v-if="detailInfo != null">
        <el-form :model="ruleForm" :rules="rules" ref="ruleForm" label-width="auto">
            <el-row :gutter="15">
                <el-col :span="8">
                    <el-form-item label="섭취주기" prop="isDaily" class="pb-4">
                        <el-switch
                            class="ml-3"
                            v-model="ruleForm.isDaily"
                            active-text="일간"
                            inactive-text="주간">
                        </el-switch>
                    </el-form-item>
                </el-col>
            </el-row>
            <el-row :gutter="15">
                <el-col :span="8">
                    <el-form-item label="1회 복용량" prop="oneTimeQuantity" class="pb-4">
                        <el-input-number class="ml-3" size="small" v-model="ruleForm.oneTimeQuantity"
                                         @change="handleChange"
                                         :min="1">
                        </el-input-number>
                    </el-form-item>
                </el-col>
            </el-row>
            <el-row :gutter="15">
                <el-col :span="8">
                    <el-form-item label="잔여량" prop="remainingQuantity" class="pb-4">
                        <el-input-number class="ml-3" size="small" v-model="ruleForm.remainingQuantity"
                                         @change="handleChange"
                                         :min="0">
                        </el-input-number>
                    </el-form-item>
                </el-col>
            </el-row>
            <el-row :gutter="15">
                <el-col :span="8">
                    <el-form-item label="복용여부" prop="isEat" class="pb-4">
                        <el-switch
                            class="ml-3"
                            v-model="ruleForm.isEat"
                            active-text="복용중"
                            inactive-text="복용하지 않음">
                        </el-switch>
                    </el-form-item>
                </el-col>
            </el-row>
            <el-row :gutter="15">
                <el-col>
                    <el-form-item>
                        <el-button type="primary" @click="doPutData()">수정</el-button>
                        <el-button @click="moveDetail(id)">취소</el-button>
                    </el-form-item>
                </el-col>
            </el-row>
        </el-form>
    </div>
</template>
```

> Template

```
<script>
export default {
    asyncData({params}) {
        return {
            id: params.id
        }
    },
    validate({params}) {
        return /^\d+$/.test(params.id)
    },
    data() {
        return {
            id: null,
            detailInfo: null,
            ruleForm: {
                isDaily: '',
                oneTimeQuantity: '',
                remainingQuantity: '',
                isEat: ''
            },
            rules: {
                oneTimeQuantity: [
                    {required: true, message: '이 값은 필수입니다.', trigger: 'blur'}
                ],
                remainingQuantity: [
                    {required: true, message: '이 값은 필수입니다.', trigger: 'blur'}
                ]
            }
        }
    },
    methods: {
        async getDetail() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            await this.$store.dispatch(this.$pillConstants.DO_DETAIL, {id: this.id})
                .then((res) => {
                    this.detailInfo = res.data.data
                    this.ruleForm.isDaily = res.data.data.isDaily
                    this.ruleForm.oneTimeQuantity = res.data.data.oneTimeQuantity
                    this.ruleForm.remainingQuantity = res.data.data.remainingQuantity
                    this.ruleForm.isEat = res.data.data.isEat
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        },
        doPutData() {
            let formName = 'ruleForm'
            this.$refs[formName].validate(valid => {
                if (valid) {
                    this.putData()
                } else {
                    return false
                }
            })
        },
```

> Script



> 구현 화면

- 1회 복용량과 잔여량은 필수값이기 때문에 validate를 이용하여 null이 되지 않도록 했습니다.
- 섭취 주기와 복용 여부는 switch를 이용하여 값이 필수로 입력되기 때문에 validate를 생략했습니다.

2-3-1. 복용 내역 목록_Java_Entity(TakeHistory)

```java
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class TakeHistory {
    no usages
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    1 usage
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pillId", nullable = false)
    private Pill pill;

    2 usages
    @Column(nullable = false)
    private LocalDate takeDate;

    2 usages
    @Column(nullable = false)
    private LocalTime takeTime;

    2 usages
    private String memo;
public static class TakeHistoryBuilder implements CommonModelBuilder<TakeHistory> {
    2 usages
    private final Pill pill;
    2 usages
    private final LocalDate takeDate;
    2 usages
    private final LocalTime takeTime;
    2 usages
    private String memo;
```

```java
public TakeHistoryBuilder(Pill pill, TakeHistoryRequest request) {
    this.pill = pill;
    this.takeDate = request.getTakeDate();
    this.takeTime = LocalTime.of(request.getTakeTimeHour(), request.getTakeTimeMinute());
}

2 usages
public TakeHistoryBuilder setMemo(String memo) {
    this.memo = memo;

    return this;
}
@Override
public TakeHistory build() { return new TakeHistory( builder: this); }
```

- TakeHistoryBuilder를 통해 Pill, takeDate, takeTime, memo를 받아옵니다.
- GeneratedValue를 통해 id가 순차적으로 생성될 수 있도록 합니다.
- Entity에 pill, takeDate, takeTime줄을 추가해 줍니다.
- 필수 값인 pill, takeDate, takeTime, memo 앞에 final을 붙여줍니다.

2-3-1. 복용 내역 목록_Java_Model,Controller(TakeHistoryRequest, PillStatusController)

```java
@Getter
@Setter
public class TakeHistoryRequest {
    no usages
    @ApiModelProperty(value = "복용 날짜(date)", required = true)
    @NotNull
    private LocalDate takeDate;

    no usages
    @ApiModelProperty(value = "복용 시간(hour)", required = true)
    @NotNull
    @Min(0)
    @Max(23)
    private Integer takeTimeHour;

    no usages
    @ApiModelProperty(value = "복용 시간(minute)", required = true)
    @NotNull
    @Min(0)
    @Max(59)
    private Integer takeTimeMinute;

    no usages
    private String memo;
}
```

```java
@Api(tags = "영양제 조회")
@RestController
@RequiredArgsConstructor
@RequestMapping("/v1/pill-state")
public class PillStatusController {
    2 usages
    private final PillTakeService pillTakeService;

    no usages
    @ApiOperation(value = "영양제 이름, 제조사, 권장 복용 수")
    @PostMapping("/take/{id}")
    public CommonResult doTake(@PathVariable long id,@RequestBody @Valid TakeHistoryRequest request) {
        pillTakeService.doTake(id, request);

        return ResponseService.getSuccessResult();
    }
```

- 1-1. PillStatusControll는 pillTakeService와 함께 작업합니다.
- 1-2. ApiModelProperty를 통해 영양제 이름, 제조사, 권장 복용 수 라고 swagger에 표시해줍니다.
- 2-1. TakeHistoryRequest에도 마찬가지로 ApiModelProperty를 복용 날짜(년, 월, 일), 복용 시간(hour), 복용 분(minute)이라고 swagger에 표시해줍니다.

2-3-1. 복용 내역 목록 (1)_Vue

```
<template>
    <div>
        <div>
            <h1 class="custom-title-h1">▶ 복용내역 목록 </h1>
        </div>
        <el-table :data="list" stripe style="width: 100%">
            <el-table-column prop="pillFullName" label="영양제 이름 및 제조사"></el-table-column>
            <el-table-column prop="eatDate" label="복용일자"></el-table-column>
            <el-table-column prop="eatTime" label="복용시간"></el-table-column>
            <el-table-column prop="memo" label="메모"></el-table-column>
        </el-table>
    </div>
</template>

<script>
export default {
    data() {
        return {
            list: [],
        }
    },
    methods: {
        getList() {
            this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, true)
            this.$store.dispatch(this.$historyConstants.DO_LIST)
                .then((res) => {
                    this.list = res.data.list
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
                .catch((err) => {
                    this.$toast.error(err.response.data.msg)
                    this.$store.commit(this.$customLoadingConstants.FETCH_LOADING_SHOW, false)
                })
        },
    },
    created() {
        this.getList()
    },
}
</script>
```

-> Template

복용내역 목록에 필요한 데이터

복용내역 리스트 메서드 생성

-> Script Method

2-3-1. 복용 내역 목록 (2)_Vue(구현)

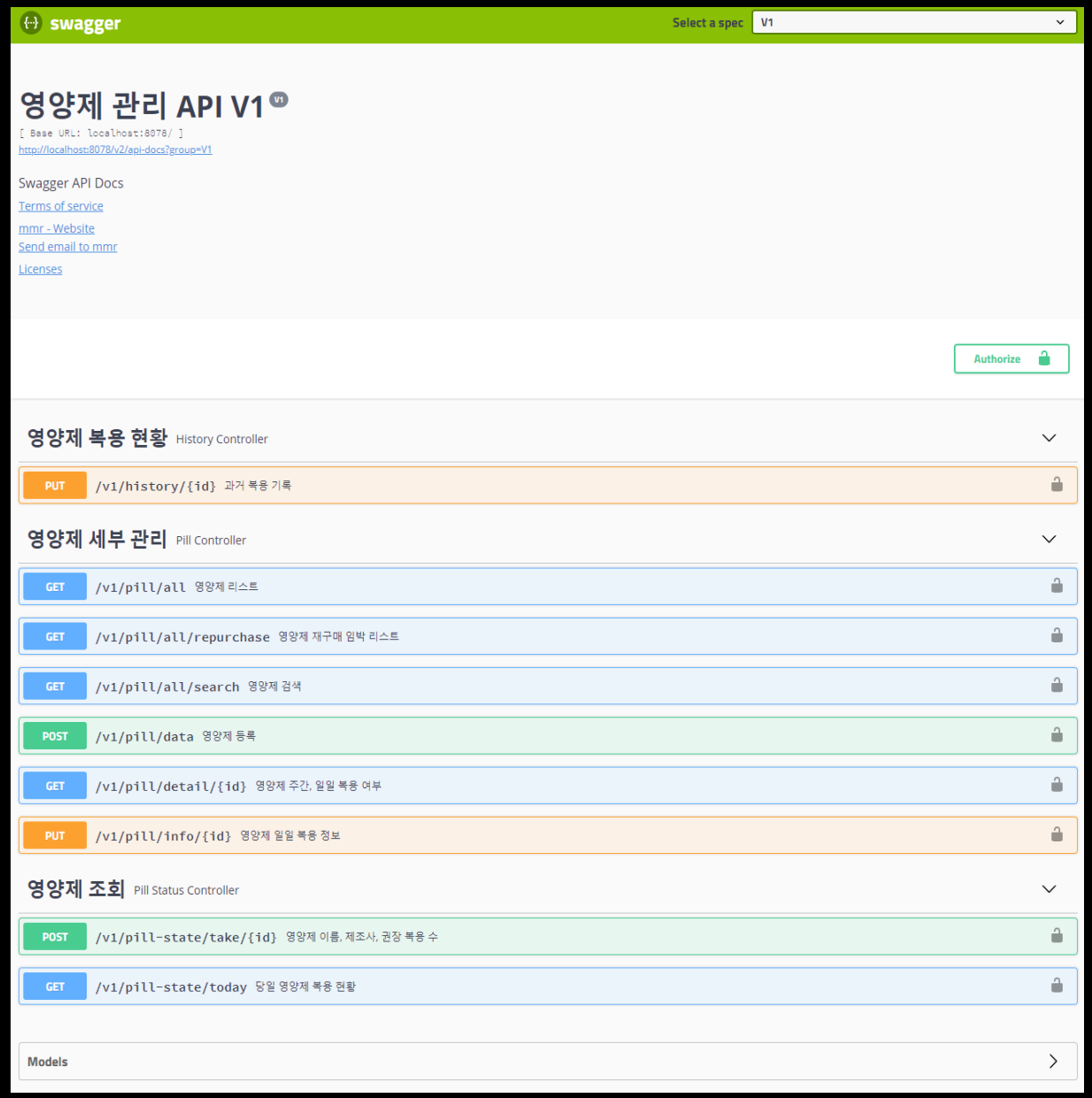
님

- HOME
- 대시보드
- 영양제 관리
- 영양제 목록
- 복용내역 관리
- 복용내역 목록

▶ 복용내역 목록

영양제 이름 및 제조사	복용일자	복용시간	메모
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-19	13:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-24	13:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-22	09:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-18	19:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-23	13:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-20	14:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-17	11:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-21	10:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-25	13:30:00	
관절엔 콘드로이친 1200 900mg 대광약품	2022-12-26	13:30:00	

Swagger-ui 화면



The End